

Documentacion Tecnica del Proyecto

Trivia Crack Quiz Multiplayer

Juego multijugador desarrollado con Elixir, Phoenix LiveView y OTP

Repositorio

https://github.com/aaabriceno/Juego_Multijugador_Paradigm_Funcional QUIZZ.git

Framework principal: Phoenix 1.8 con LiveView

Paradigma concurrente: Modelo de actores sobre Erlang/OTP

Documento complementario: docs/guia_instalacion_local.md

Junio 2026

Índice

1. Vision general del proyecto	2
2. Arquitectura general y estructura del juego	2
2.1. Capas principales	3
2.2. Arbol de supervision	3
3. Estrategia de concurrencia y coordinacion en tiempo real	4
3.1. Una sala equivale a un proceso	5
3.2. Mensajes sincronizados y consistencia	5
3.3. PubSub para propagacion en tiempo real	5
3.4. Temporizadores y eventos internos	5
3.5. Monitoreo de jugadores y tolerancia a fallos	5
4. Definicion de modulos y funciones principales	6
4.1. Creacion y administracion de salas	6
4.2. Ciclo de vida de la partida	7
4.3. Seleccion de preguntas, filtros y validacion de respuestas	7
4.4. Publicacion de estado a los clientes	8
5. Decisiones funcionales y de diseno	8
6. Flujo operativo de una partida	8
7. Conclusion	9

1. Vision general del proyecto

Trivia Crack Quiz Multiplayer es una aplicacion web multijugador en tiempo real que permite crear salas independientes de trivia, unir jugadores desde varios navegadores y mantener el puntaje sincronizado sin recargas de pagina. El proyecto se construyo con Elixir y Phoenix LiveView, y adopta el modelo de actores de Erlang/OTP para que cada sala de juego exista como un proceso aislado con estado propio.

El sistema cubre tres objetivos tecnicos centrales:

- administrar varias salas en paralelo sin compartir memoria entre ellas;
- mantener una experiencia en tiempo real mediante PubSub y LiveView;
- separar la logica funcional pura del juego de los efectos asociados a procesos, temporizadores, supervision y comunicacion con clientes.

Desde la perspectiva del usuario, el flujo principal se compone de cuatro pantallas: lobby general, creacion de sala, partida activa y tablero espectador. Desde la perspectiva tecnica, esas vistas son clientes LiveView que delegan toda la coordinacion del juego a procesos OTP y a un motor funcional de reglas.

2. Arquitectura general y estructura del juego

La aplicacion sigue la organizacion habitual de un proyecto Phoenix: el dominio del juego vive en `lib/trivia_crack_quiz/` y la capa web en `lib/trivia_crack_quiz_web/`. La Figura 1 resume como se relacionan sus piezas principales.

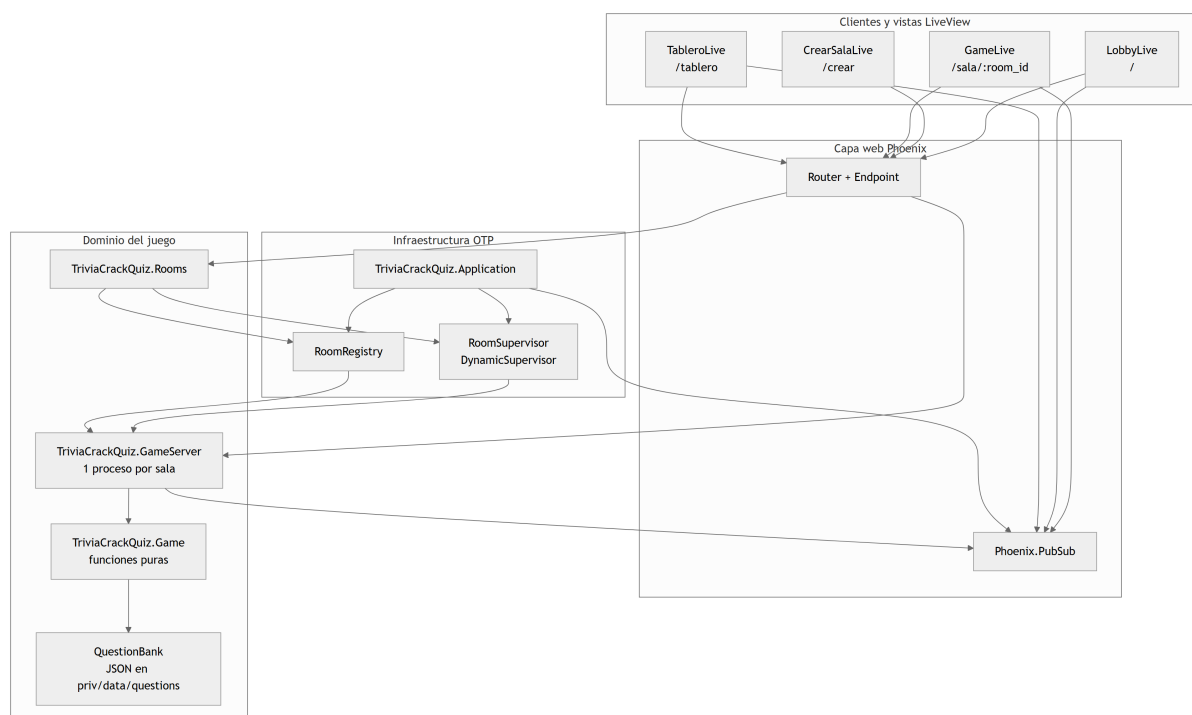


Figura 1: Estructura general del juego y dependencias entre sus componentes.

2.1. Capas principales

- **Capa de interfaz:** los módulos `LobbyLive`, `CrearSalaLive`, `GameLive` y `TableroLive` renderizan la interfaz y capturan eventos del navegador.
- **Capa de coordinacion:** `Rooms` crea, lista y cierra salas utilizando `DynamicSupervisor` y `Registry`.
- **Capa de procesos por sala:** `GameServer` mantiene el estado consistente de una partida, programa temporizadores y comunica cambios a los clientes.
- **Capa funcional pura:** `Game` concentra las reglas del juego como transformaciones de estado inmutables.
- **Datos de preguntas:** `QuestionBank` carga y normaliza el banco de preguntas desde `priv/data/questions/`.

2.2. Arbol de supervision

El módulo `TriviaCrackQuiz.Application` inicia los procesos globales que permiten que la aplicación funcione:

- `TriviaCrackQuizWeb.Telemetry`
- `DNSCluster`

- Phoenix.PubSub
- TrivialCrackQuiz.RoomRegistry
- TrivialCrackQuiz.RoomSupervisor
- TrivialCrackQuizWeb.Endpoint

La estrategia `:one_for_one` garantiza que, si un proceso falla, solo se reinicia ese proceso y no toda la aplicacion. Esta decision es esencial porque evita que una falla en una sala interrumpa a las demas partidas activas.

3. Estrategia de concurrencia y coordinacion en tiempo real

La estrategia de concurrencia del proyecto se apoya por completo en Erlang/OTP y en el paso de mensajes. En lugar de compartir un estado global mutable, cada sala se representa como un proceso `GenServer` independiente identificado por su `room_id`. La Figura 2 describe el flujo de eventos y actualizaciones dentro de una sala.

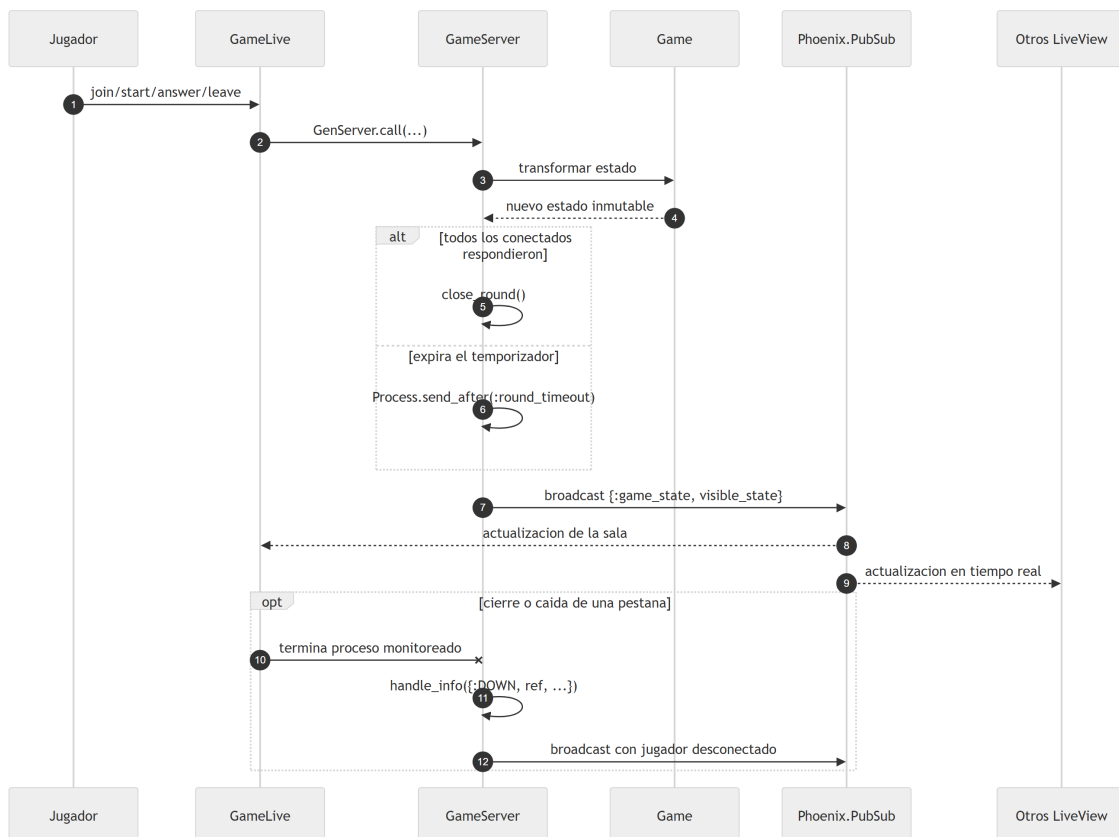


Figura 2: Secuencia de concurrencia para una sala de juego.

3.1. Una sala equivale a un proceso

Cuando un usuario crea una sala, `Rooms.create/2` solicita al `RoomSupervisor` que inicie un hijo `GameServer`. Ese hijo queda registrado en `RoomRegistry` mediante una clave unica basada en el `room_id`. A partir de ese momento:

- todas las operaciones de la sala se encaminan al mismo proceso;
- el estado de la partida solo existe dentro de ese `GenServer`;
- otras salas no pueden corromper ese estado porque no comparten memoria.

3.2. Mensajes sincronizados y consistencia

La capa web no modifica el estado del juego directamente. Los eventos del usuario se traducen a llamadas como `join`, `start`, `answer`, `leave` o `reset`, que llegan al `GameServer` mediante `GenServer.call/2`. Como un `GenServer` atiende un mensaje a la vez, el estado se actualiza secuencialmente y se evita la necesidad de bloqueos explicitos.

3.3. PubSub para propagacion en tiempo real

Despues de cada cambio, `GameServer` publica el estado visible mediante `PubSub.broadcast/3`. Los `LiveView` suscritos al topico de esa sala reciben el evento `:game_state` y re-renderizan la interfaz. El lobby general utiliza otro topico, `rooms`, para enterarse de altas, bajas o cambios de estado de las salas.

3.4. Temporizadores y eventos internos

El flujo de la partida no depende solo de mensajes del usuario. El proceso de la sala programa eventos internos con `Process.send_after/3` para tres casos:

- **fin de ronda:** dispara `:round_timeout` cuando se agota el tiempo de respuesta;
- **pantalla de resultados:** dispara `:results_timeout` para avanzar automaticamente a la siguiente pregunta;
- **sala vacia:** dispara `:empty_room_timeout` cuando no queda ningun jugador conectado.

Esta estrategia evita depender de cron externos y concentra toda la logica temporal dentro del actor que ya administra el estado de la sala.

3.5. Monitoreo de jugadores y tolerancia a fallos

Cada vez que un jugador entra o se reconecta, el `GameServer` monitorea el proceso `LiveView` asociado. Si la pestana se cierra o el proceso termina, el servidor recibe un mensaje `{:DOWN, ref, :process, pid, reason}` y marca al jugador como desconectado. Este comportamiento permite:

- conservar el puntaje del jugador aunque refresque la pagina;
- no bloquear la partida esperando respuestas de clientes desconectados;
- cerrar automaticamente una sala abandonada sin afectar al resto del sistema.

4. Definicion de modulos y funciones principales

Las funciones importantes del sistema se organizan por responsabilidad, no por orden de archivo. Los modulos mas relevantes son los siguientes:

TriviaCrackQuiz.Application Arranque OTP. Define el arbol de supervision, inicia PubSub, Registry, DynamicSupervisor y el Endpoint web.

TriviaCrackQuiz.Rooms Gestor de salas. Crea, lista, cierra y busca salas; tambien notifica al lobby cuando cambia el conjunto de partidas activas.

TriviaCrackQuiz.GameServer Actor por sala. Mantiene el estado de una partida, procesa mensajes, controla timers y difunde actualizaciones en tiempo real.

TriviaCrackQuiz.Game Motor funcional puro. Implementa reglas del juego: altas de jugadores, rondas, filtros, puntaje, respuestas correctas y cierre de partida.

TriviaCrackQuiz.QuestionBank Proveedor de datos. Carga y normaliza preguntas desde archivos JSON del proyecto.

TriviaCrackQuizWeb.GameLive Vista interactiva. Presenta el estado de la sala y traduce acciones del navegador en eventos de servidor.

4.1. Creacion y administracion de salas

Rooms concentra la administracion del ciclo de vida de las salas:

- **create/2**: arranca una sala con filtros opcionales y devuelve un **room_id** estable.
- **create_random/1**: genera un identificador legible y crea una sala nueva.
- **create_named/2**: transforma un nombre escrito por el usuario en un identificador apto para URL.
- **random_open/1**: localiza una sala en espera con filtros compatibles o crea una nueva si no existe.
- **list/0**: construye un resumen de las salas activas para el lobby y el tablero espectador.
- **close/1**: termina el proceso de una sala cuando deja de ser necesaria.

Estas funciones encapsulan el uso de **DynamicSupervisor** y **Registry**, de modo que el resto del sistema nunca trabaja con pids de forma manual.

4.2. Ciclo de vida de la partida

El modulo `Game` define la logica del juego como transformaciones puras de un mapa de estado:

- `new_state/2`: crea el estado inicial con filtros, tiempos, ronda sorpresa y banco de preguntas disponible.
- `add_player/3` y `remove_player/2`: agregan o eliminan jugadores sin romper la inmutabilidad del estado.
- `start/1`: valida que exista el minimo de jugadores e inicia la primera ronda.
- `next_question/1`: selecciona una pregunta, actualiza la ronda actual y reinicia respuestas pendientes.
- `register_answer/3`: registra la respuesta de un jugador junto con el instante exacto en que respondio.
- `evaluate_round/1`: decide si cada respuesta es correcta, calcula puntaje y genera la estructura de resultados de la ronda.
- `reset/1`, `reopen_room/1` y `finish/1`: reinician o cierran la partida segun el estado alcanzado.

La funcion `visible_state/1` cumple un papel de seguridad importante: antes de enviar el estado al navegador elimina respuestas correctas, timers, monitores y datos internos que no deben exponerse al cliente.

4.3. Seleccion de preguntas, filtros y validacion de respuestas

El juego no usa una base de datos; las preguntas se cargan desde archivos JSON en `priv/data/questions/`. Las funciones clave para esta parte son:

- `QuestionBank.load_questions/0`: lee y normaliza el banco de preguntas del proyecto.
- `Game.available_categories/1` y `available_types/1`: extraen las opciones disponibles para los formularios de creacion de sala.
- `Game.correct_answer?/2`: compara la respuesta enviada por el usuario contra la respuesta oficial y alternativas aceptadas.
- `Game.question_time_ms/1`: ajusta la duracion de la ronda segun el tipo de pregunta.

En preguntas de tipo `:quick_answer`, la validacion tolera errores de tipeo pequenos mediante una implementacion propia de distancia de Levenshtein. Esta decision mejora la experiencia de juego sin desordenar la capa de presentacion.

4.4. Publicacion de estado a los clientes

`GameServer` conecta la logica funcional con la infraestructura concurrente:

- `join/3`, `reconnect/2`, `leave/2`, `start_game/1`, `answer/3` y `reset/1` representan la API publica de una sala.
- `handle_call/3` serializa las operaciones solicitadas por los clientes y delega cada decision de negocio al modulo `Game`.
- `handle_info/2` procesa expiracion de timers y eventos `:DOWN` provenientes del monitoreo de jugadores.
- `broadcast_state/1` publica la vista saneada del estado mediante PubSub para actualizar clientes y tablero.

5. Decisiones funcionales y de diseno

El proyecto fue construido bajo lineamientos del paradigma funcional, y eso se refleja en varias decisiones de implementacion:

- **Inmutabilidad:** las reglas del juego no mutan estructuras en el lugar; siempre devuelven un nuevo mapa de estado.
- **Funciones puras:** `TriviaCrackQuiz.Game` no conoce sockets, procesos ni PubSub. Solo transforma datos.
- **Separacion de efectos:** `GameServer` concentra IO, temporizadores, monitoreo y mensajeria, mientras el motor funcional decide reglas.
- **Pattern matching y guards:** el flujo del programa distingue estados validos directamente en las cabeceras de funciones, por ejemplo al evaluar rondas o procesar timeouts.
- **Paso de mensajes en lugar de locks:** la coherencia se garantiza al procesar un mensaje por vez dentro del actor de la sala.

Esta separacion mejora la mantenibilidad: las pruebas del dominio pueden ejecutarse sin levantar la interfaz web, mientras que la capa concurrente puede probarse como comportamiento de actores reales.

6. Flujo operativo de una partida

El comportamiento del sistema puede resumirse en la siguiente secuencia:

1. Un jugador entra al lobby y crea una sala o solicita union aleatoria.
2. `Rooms` crea o localiza el `GameServer` correspondiente.

3. **GameLive** se suscribe al topico `game:room:<room_id>`.
4. Los jugadores se registran, el servidor monitorea sus procesos y el lobby recibe notificaciones de cambio de aforo.
5. Cuando hay al menos tres jugadores, uno de ellos inicia la partida.
6. Cada respuesta entra al **GameServer**, que actualiza el estado mediante funciones puras y recalcula si la ronda debe cerrarse.
7. El proceso difunde el nuevo estado y todos los clientes de la sala ven el mismo resultado en tiempo real.
8. Tras la ronda final, la partida muestra podio y ganador, y puede reiniciarse o cerrarse.

7. Conclusion

La arquitectura del proyecto demuestra que es posible construir un juego multijugador coherente y extensible usando principios funcionales y concurrencia basada en actores. El uso de Phoenix LiveView reduce la complejidad de sincronizacion en el cliente, mientras que OTP ofrece aislamiento de fallos, supervision y procesos ligeros para modelar cada sala como una unidad independiente.

El resultado es un sistema adecuado para entregas academicas y para evolucionar tecnicamente: admite nuevas categorias de preguntas, ajustes de puntaje, persistencia futura o nuevas vistas sin romper la separacion entre dominio, procesos y presentacion.

Nota de mantenimiento. Este documento se complementa con la guia ubicada en `docs/guia_instalacion_local.md`, donde se detallan los pasos para clonar el repositorio, ejecutar el proyecto y regenerar el PDF de esta documentacion tecnica.